# AI PROTOTYPING HACKATHON

BEGINNER MANUAL | INTERNAL HACKATHON 2026

From real-life pain point to a safe, measurable prototype.

START HERE

# How to use this manual

You do not need to be a data scientist. Start with a real work problem, choose the smallest toolset that can demonstrate value, and keep a person in control.

| 1. Frame | 2. Build | 3. Prove |
|---|---|---|
| Name the problem, user, current process, and one measurable outcome. | Create the smallest end-to-end path that works with safe sample data. | Test quality, calculate value and cost, document risks, then demo. |

## Choose your route

| | | |
|---|---|---|
| moves approvals, alerts, files, or forms | Power Apps + Power Automate | Form, workflow, Teams/email alert |
| shows trends, exceptions, or KPIs | Excel/CSV + Power BI | Dashboard with filters and insights |
| predicts, classifies, clusters, or detects anomalies | Colab/Kaggle + Python | Notebook, model result, simple chart |
| answers questions over approved documents | Approved AI assistant or RAG prototype | Cited Q&A; experience with guardrails |

**Golden rule**

Use the least sensitive data and the simplest architecture that can prove the idea. A small working prototype is stronger than a large diagram that does not run.

HACKATHON FINISH LINE

# What a strong submission contains

The intro deck asks for both a business layer and a technical layer. Your demo should connect them.

| Business layer | Technical layer | Evidence |
| --- | --- | --- |
| Problem statement | Data | Working prototype |
| Target user | Model or method | Impact estimate |
| Business value proposition | Workflow and architecture | Cost vs benefit |
| | Evaluation and safety | Pilot next step |

- State the problem and user in one sentence.
- Show the current process and its gap.
- Name the dataset or documents used.
- Explain the AI method or workflow in plain language.
- Demonstrate a working path, including a failure or edge case.
- Quantify time saved, cost avoided, quality improved, or turnaround reduced.
- Flag privacy, legal, compliance, and policy considerations early.
- End with a realistic pilot plan.

**Prototype, not production**

A hackathon prototype may use sample data and manual steps behind the scenes. Label those limitations honestly; do not present them as production-ready automation.

STEP 1

# Frame the problem before opening a tool

| | | |
|---|---|---|
| Problem | When [user] does [task], [pain] causes [measurable consequence]. | We want an AI chatbot. |
| User | The person who performs, approves, or receives the work. | Everyone. |
| Baseline | Today it takes X minutes, Y handoffs, or has Z% rework. | It is slow. |
| Target | Reduce cycle time from X to Y while maintaining quality threshold Q. | Make it better. |
| Boundary | The prototype recommends; a named role approves before action. | The AI decides. |

## Copy-and-fill problem statement

For [target user], the current [process] requires [time/effort] and often causes [error/delay]. We will prototype [assistant/workflow/dashboard/model] that uses [safe data] to [action]. Success means [metric and target], with [human role] approving [high-impact step].

STEP 2

# Pick the smallest useful tool stack

| | | | |
|---|---|---|---|
| Microsoft Copilot / approved chat | Drafting, summarizing, ideation, structured extraction | Ask for a table, checklist, or options; verify every claim | Do not paste restricted data; outputs can be wrong |
| Excel | Small datasets, calculations, quick charts, test cases | Create a clean table with one header row | Mixed units, hidden blanks, personal data |
| Power Apps | Simple user-facing forms and canvas apps | Create a canvas app from a safe list/table | Environment, connector, and sharing permissions |
| Power Automate | Triggers, approvals, notifications, file movement | Build one trigger -> one action -> one confirmation | Loops, duplicate runs, connector permissions |
| Power BI | Interactive dashboards and KPI storytelling | Import a clean CSV; build three visuals and one slicer | Misleading scales and unclear measures |
| Google Colab | Browser-based Python notebooks | Upload a safe CSV; inspect with pandas | Session resets and public-cloud data restrictions |
| Kaggle | Public datasets and hosted notebooks | Read the Data Card and license before use | Quality, license, and hidden leakage |
| GitHub | Versioning and team collaboration | Use a private approved repository and clear README | Never commit secrets, tokens, or restricted data |

# Prompting that produces usable work

A prompt is a work brief. Give the model a role, goal, context, constraints, and output format - then review the result.

## The CLEAR pattern

| | | |
|---|---|---|
| C - Context | Audience, process, data meaning | This is a weekly maintenance exception report for site supervisors. |
| L - Limit | Scope, policy, length, forbidden actions | Use only the supplied rows. Do not infer missing causes. |
| E - Expected output | Format and fields | Return a 5-column table: asset, issue, severity, evidence, next step. |
| A - Ask | The exact task | Group repeated issues and prioritize the top five. |
| R - Review | Checks and uncertainty | Cite row IDs; mark uncertain items; list assumptions. |

## Copyable prompt

You are helping [role]. Goal: [task]. Context: [facts]. Use only [approved inputs]. Constraints: [policy, length, tone]. Return [format]. For every recommendation, show [evidence]. If information is missing, say so. Before finalizing, check [quality tests].

**Never trust fluent wording as proof**

Verify names, numbers, calculations, citations, policy statements, and technical claims against an authoritative source or test case.

DATA SAFETY GATE

# Decide what may enter each tool

Tool access does not automatically mean data is approved for that tool. Follow NESR policy and ask the data owner or security/legal contact when unsure.

| | | |
|---|---|---|
| Public | Open government data, public Kaggle dataset with suitable license | Document source, license, version, and limitations. |
| Synthetic | Invented records that mimic structure but not real people/assets | Preferred for demos; state how it was generated. |
| Internal | Company process data without highly sensitive fields | Use only approved corporate tools and least-privilege access. |
| Restricted / personal / confidential | Employee identifiers, customer data, credentials, contracts, sensitive operations | Do not use without explicit approval and controls. Create a synthetic substitute. |

## Before importing or pasting data

- Who owns the data, and have they approved this use?
- Does it include names, IDs, contact details, location, contracts, or operationally sensitive fields?
- Is the tool approved for that classification and region?
- Can you remove columns, aggregate rows, or use synthetic data?
- How will files, prompts, logs, and outputs be deleted after the event?
- Who can access the prototype and shared links?

LOW-CODE ROUTE

# Build a simple workflow with Power Automate

Best for repetitive handoffs: a form arrives, a rule runs, someone approves, and a notification or record is created.

## Your first cloud flow

- Write the flow on paper: trigger -> checks -> action -> confirmation.
- Open Power Automate in the correct corporate environment.
- Choose an automated, instant, or scheduled cloud flow.
- Start with one safe trigger, such as a new form response or approved list item.
- Add one condition. Name branches in plain language: Approved / Needs review.
- Add one action, such as create an item or send a Teams/email notification.
- Test with three records: normal, missing value, and boundary case.
- Open run history and capture evidence of success and failure handling.

| | |
|---|---|
| What starts it? | A named event, not 'when something happens.' |
| Can it run twice? | Use an ID/status field and prevent duplicate processing. |
| What if a connector fails? | Notify an owner and preserve the source record. |
| Where is human approval? | Before external messages, payments, access changes, or high-impact decisions. |

**Licensing and permissions**

Connectors, AI features, environments, and sharing may require admin enablement or specific licenses. Confirm access early; keep a fallback demo video or mocked step.

LOW-CODE ROUTE

# Build a front end with Power Apps

Use Power Apps when a user needs to enter, review, filter, or approve information through a simple interface.

## Minimum viable canvas app

- Sketch three screens: Home/List, Details, and Submit/Review.
- Connect to a safe, approved source such as SharePoint, Dataverse, or a small prototype table.
- Use clear field labels and mark required fields.
- Validate input before submission; show a useful error message.
- Add a status field so users know Draft, Submitted, Approved, or Returned.
- Use role-based visibility only as a usability aid; secure the underlying data source too.
- Test on the device size you will demo.
- Share only with the intended test group.

| Good prototype | Avoid | Demo proof |
| --- | --- | --- |
| One job, few screens, obvious next action, safe sample data. | A mini enterprise system with ten screens and no tested end-to-end path. | Submit one record, show where it is stored, then show the downstream flow. |

ANALYTICS ROUTE

# Turn a clean table into a Power BI story

A dashboard should help a user notice, decide, and act - not merely display many charts.

## First report recipe

- Define one decision: What should the viewer do differently after seeing this?
- Create a tidy table: one row per event/entity, one header row, consistent dates and units.
- Import the file and inspect data types in Power Query.
- Create three measures: volume, outcome, and change versus baseline.
- Build three visuals: headline KPI, trend, and breakdown.
- Add one slicer that matters to the decision, such as site, category, or month.
- Use titles that state the question: 'Where are delays increasing?'
- Check filters, blank values, totals, and axis scales against the source.

## Simple measures to prepare

| | |
|---|---|
| Cycle time saved | (baseline minutes - new minutes) x monthly cases |
| Error reduction | baseline error rate - prototype error rate |
| Cost avoided | hours saved x loaded hourly cost |
| Coverage | records successfully processed / eligible records |

NOTEBOOK ROUTE

# Start safely in Google Colab

A notebook mixes runnable code, results, charts, and explanation. It is ideal for a transparent data-science prototype.

## Five-cell starter workflow

```
# 1. Import libraries
import pandas as pd
import matplotlib.pyplot as plt

# 2. Load a SAFE local CSV
from google.colab import files
uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

# 3. Inspect before modeling
display(df.head())
print(df.shape)
print(df.dtypes)
print(df.isna().sum())

# 4. Summarize
display(df.describe(include='all').T)

# 5. Save a result
df.to_csv('prototype_output.csv', index=False)
```

- Rename cells with Markdown headings so teammates can follow the story.
- Run from top to bottom before the demo; hidden state causes surprises.
- Pin dependency versions only if you install extra packages.
- Download a copy of the notebook and outputs; hosted sessions can reset.
- Do not put secrets directly in code cells or shared notebooks.

NOTEBOOK ROUTE

# Find and assess a Kaggle dataset

The fastest dataset is not always the safest or most useful. Read its documentation before downloading.

## Dataset fitness checklist

- The license permits your intended use.
- The Data Card explains columns, units, source, collection period, and known limitations.
- The target or outcome is actually present and meaningful.
- The dataset is recent enough for the question.
- Missingness and class imbalance are manageable.
- Rows are not duplicated across training and testing.
- It does not contain personal or sensitive data you do not need.
- You record the dataset URL, owner, version, and access date.

## Optional download with kagglehub

```
# In Colab or another Python environment
!pip -q install kagglehub
import kagglehub

# Replace with the dataset handle from its URL
path = kagglehub.dataset_download('owner/dataset-name')
print(path)
```

**Prefer attachment inside Kaggle notebooks**

Kaggle's official kagglehub library can attach resources directly in Kaggle notebooks. Authentication may be required elsewhere. Never expose an API token in a shared notebook or repository.

NOTEBOOK ROUTE

# Clean and understand data with pandas

## A repeatable inspection block

```
# Standardize column names
df.columns = (df.columns.str.strip().str.lower()
              .str.replace(' ', '_', regex=False))

# Remove exact duplicates
df = df.drop_duplicates()

# Parse a date and numeric field
df['event_date'] = pd.to_datetime(df['event_date'], errors='coerce')
df['value'] = pd.to_numeric(df['value'], errors='coerce')

# Review quality
quality = pd.DataFrame({
    'dtype': df.dtypes.astype(str),
    'missing': df.isna().sum(),
    'unique': df.nunique()
})
display(quality)
```

| | | |
|---|---|---|
| Numbers stored as text | Currency signs, commas, mixed labels | Clean deliberately; count values that become missing. |
| Very high accuracy | Target leakage or duplicate entities | Audit features; split by time/entity where appropriate. |
| Model ignores a group | Imbalance or missing coverage | Report group counts and per-group performance. |
| Beautiful trend, wrong conclusion | Seasonality or changing denominator | Compare like periods and expose the denominator. |

AI/MODEL ROUTE

# Build a baseline before a clever model

A baseline tells judges whether AI adds value. Use a simple rule or common model first, then compare fairly.

| | | |
|---|---|---|
| Classification | Most-common class or logistic regression | Precision, recall, F1, confusion matrix |
| Regression | Mean/median or linear regression | MAE, RMSE; compare with business tolerance |
| Forecasting | Last value or seasonal average | MAE/MAPE with caveats for near-zero values |
| Anomaly detection | Threshold or interquartile range | True alerts, missed events, false alarms |
| Document Q&A; | Keyword search or curated FAQ | Answer correctness, citation correctness, abstention rate |

## Evaluation discipline

- Separate training/tuning data from final test data.
- Split by time or entity when random splitting would leak future or repeated information.
- Report the baseline beside the prototype result.
- Show the confusion matrix or several real examples, not only one score.
- Define the cost of false positives and false negatives with the business owner.
- Test missing inputs, unusual values, and groups underrepresented in the data.
- Record model, parameters, dataset version, and random seed where applicable.

DOCUMENT Q&A;

# Know when you need RAG

Retrieval-augmented generation (RAG) first finds relevant approved passages, then asks a language model to answer from those passages.

| 1. Retrieve | 2. Generate | 3. Verify |
| --- | --- | --- |
| Search indexed document chunks using keywords or embeddings. | Give the selected passages and question to the model. | Show citations, test unsupported questions, and allow abstention. |

## Use RAG when

- Answers must rely on a changing or organization-specific document collection.
- Users need to see where an answer came from.
- The documents are approved for the selected platform and access controls.

## Do not use RAG as a shortcut when

- A simple search page or FAQ is enough.
- Documents have conflicting owners, versions, or classifications.
- The system cannot enforce the user's document permissions.

| | |
| --- | --- |
| Answerable question | Correct answer with the correct supporting passage. |
| Unanswerable question | Says it cannot find support; does not invent. |
| Conflicting documents | Surfaces the conflict and dates/sources. |
| Permission boundary | A user cannot retrieve content they cannot normally access. |

ARCHITECTURE

# Draw the end-to-end path

Use boxes and arrows that a nontechnical judge can follow. Every box should have an owner, input, output, and failure behavior.

```
USER / TRIGGER -> INPUT -> VALIDATE -> AI OR RULE -> HUMAN REVIEW -> ACTION -> LOG / KPI
```

| | |
|---|---|
| User | Who starts the process and who receives the result? |
| Data | Where does it come from? What is its classification and retention? |
| Logic / AI | What rule, model, prompt, or retrieval step is used? |
| Integration | Which connectors, APIs, files, or services move information? |
| Human control | Who approves, corrects, overrides, or stops the action? |
| Monitoring | What is logged? Which quality, cost, and safety KPIs are watched? |
| Failure | What happens when data is missing, a model is uncertain, or a service is down? |

**Label prototype shortcuts**

If a teammate manually copies output between tools, show that honestly as a dashed 'manual for prototype' step and describe how a pilot would integrate it.

IMPACT

# Quantify the value; do not just claim it

| | | |
|---|---|---|
| Time saved | (before minutes - after minutes) x cases per month | Timed sample, workflow logs, user observation |
| Cost avoided | hours saved x loaded hourly cost | Finance-approved assumption or clearly labeled estimate |
| Quality improved | baseline defect rate - prototype defect rate | Blind review or held-out test cases |
| Faster turnaround | baseline median duration - prototype median duration | Comparable cases and consistent start/end points |
| Value created | total benefit - total recurring cost | Scenario range: conservative, expected, optimistic |

## Worked example

A review takes 18 minutes today and 11 minutes with the prototype. At 400 cases/month: 7 x 400 = 2,800 minutes = 46.7 hours saved/month. If the loaded labor rate is AED 180/hour, estimated gross value is AED 8,406/month. If monthly compute, licensing, maintenance, and human review cost AED 2,700, estimated net value is AED 5,706/month. Show assumptions and sensitivity; do not present estimates as audited savings.

COST

# Prove that the technical solution is worth running

| | |
|---|---|
| AI / compute | Model usage, API calls, notebook/GPU time, hosting, storage |
| Licensing | Premium connectors, Power Platform capacity, user licenses |
| Human review | Time to check, approve, and correct AI output |
| Build / integration | Developer or analyst time, connectors, testing |
| Maintenance | Monitoring, retraining, prompt/document updates, support |
| Risk controls | Security review, audit logging, access management |

## Use a range, not false precision

- Conservative case: lower adoption and benefit, higher cost.
- Expected case: best defensible assumptions.
- Optimistic case: higher adoption, but still plausible.
- State which costs are one-time versus monthly.
- Name who must validate pricing, labor rates, and licensing before a pilot.

# Build controls into the prototype

| | | |
|---|---|---|
| Privacy / confidentiality | Minimize fields; synthetic data; approved platform; restricted sharing | Data inventory and classification note |
| Hallucination / error | Ground in sources; validation rules; abstain when uncertain | Incorrect and unsupported test cases |
| Bias / unfair impact | Check coverage and outcomes across relevant groups | Group counts and performance review |
| Automation harm | Human approval before consequential action; easy override | Approval screen and audit trail |
| Prompt injection / unsafe content | Treat external text as data, restrict tools/actions, validate output | Malicious or irrelevant input test |
| Legal / policy mismatch | Identify applicable rules and obtain owner review | Open questions and required approvals |

## Safety card to include in your submission

Intended use: [what it supports]. Not for: [forbidden/high-risk uses]. Data: [source/classification]. Known limitations: [where it fails]. Human control: [who reviews]. Monitoring: [quality/cost/safety metrics]. Escalation: [who is contacted and when].

TESTING

# Use a small but deliberate test pack

| | | |
|---|---|---|
| Happy path | Complete, normal input | Correct output and logged success |
| Missing field | No date, category, or document | Helpful validation; no silent guessing |
| Boundary | Zero, maximum, deadline, or threshold value | Correct rule at the exact edge |
| Messy input | Typo, duplicate, unexpected format | Normalize, reject, or route for review |
| Unsupported request | Question outside documents/scope | Abstain and direct user appropriately |
| Adversarial | Instruction hidden in uploaded text | Ignore unsafe instruction; preserve control boundary |
| Service failure | Connector/model unavailable | Safe fallback; owner notified; no lost record |

## Test log columns

```
Test ID | Input | Expected result | Actual result | Pass/Fail | Evidence link | Owner | Fix/retest date
```

**Freeze a demo version**

After final testing, duplicate or tag the working version. Last-minute edits are a common cause of failed demos.

DEMO

# Tell a six-minute evidence story

| | |
|---|---|
| 0:00-0:45 | Problem, user, current process, and baseline. |
| 0:45-1:15 | Architecture and data classification in one slide. |
| 1:15-3:15 | Live happy path from input to outcome. |
| 3:15-4:00 | One edge case, failure, or human approval control. |
| 4:00-4:45 | Evaluation results and limitations. |
| 4:45-5:30 | Impact math and cost vs benefit. |
| 5:30-6:00 | Pilot plan, owner, approvals, and next milestone. |

## Demo resilience checklist

- Use a clean test account and approved sample data.
- Preload slow pages but still explain the actual trigger.
- Keep a 60-90 second backup recording.
- Export key screenshots/results in case a service is unavailable.
- Remove notifications, credentials, unrelated tabs, and sensitive history.
- Assign one narrator, one operator, and one teammate to track time/questions.

TEAM PLAN

# A practical 2-day build sequence

| | |
|---|---|
| Hour 0-1 | Agree on user, problem, baseline, target metric, and data boundary. |
| Hour 1-2 | Choose one route; sketch architecture and happy path. |
| Hour 2-5 | Clean/create sample data; build the thinnest end-to-end prototype. |
| Hour 5-7 | Add one useful feature only after the happy path works. |
| Hour 7-9 | Test normal, missing, boundary, unsupported, and failure cases. |
| Hour 9-11 | Measure quality; calculate impact and cost scenarios. |
| Hour 11-13 | Document controls, limitations, and pilot requirements. |
| Hour 13-15 | Build the pitch; rehearse with a timer and skeptical questions. |
| Final hour | Freeze version, verify links/access, save backup video and exports. |

| Business lead | Builder | Data/test lead |
|---|---|---|
| Problem, user, baseline, value, pitch. | Prototype, architecture, runbook, backup. | Data quality, evaluation, risk tests, evidence. |

TROUBLESHOOTING

# When the prototype does not cooperate

| | | |
|---|---|---|
| Cannot access a connector or AI feature | Check account, environment, license, admin policy | Mock that step and document pilot dependency |
| Flow runs repeatedly | Inspect trigger condition and status/idempotency field | Disable flow; demo with a controlled instant trigger |
| App user sees no data | Check source sharing and row permissions | Use a controlled demo account and safe sample source |
| Notebook lost state | Restart and Run all; verify file paths/install cells | Keep downloaded notebook, data, and output copies |
| Model score is suspiciously high | Audit leakage, duplicates, split method | Return to baseline and honest limitation |
| AI answer changes each run | Constrain prompt, temperature if available, source context, output schema | Use fixed test examples and human review |
| Live demo internet failure | Do not improvise with sensitive local files | Play backup video and show exported evidence |

# Copy these into your working document

## One-page concept

Project title: ____
Target user: ____
Problem and current baseline: ____
Proposed prototype: ____
Data source and classification: ____
AI/model/workflow: ____
Success metric and target: ____
Human control: ____
Known limitations: ____
Pilot owner and next step: ____

## Cost vs benefit

Monthly cases: ____ | Minutes saved/case: ____ | Hours saved: ____ | Value/hour: ____ | Gross benefit: ____ | AI/compute: ____ | Licensing: ____ | Human review: ____ | Maintenance: ____ | Net value: ____

## Architecture inventory

```
Component | Owner | Input | Output | Data class | Access | Failure behavior | Log/KPI
```

## Decision log

```
Date | Decision | Options considered | Reason | Owner | Assumption to validate
```

GLOSSARY

# Plain-English AI and technical terms

| | |
|---|---|
| AI model | A learned system that maps inputs to predictions or generated outputs. |
| LLM | A large language model trained to predict and generate text-like sequences. |
| Prompt | Instructions and context sent to a generative AI model. |
| Hallucination | A plausible-sounding output that is unsupported or incorrect. |
| Token | A chunk of text processed by a language model; not always a full word. |
| Embedding | A numeric representation used to compare meaning or similarity. |
| RAG | Retrieval-augmented generation: retrieve sources, then answer from them. |
| Agent | A system that uses a model plus tools and control logic to take steps. |
| Feature | An input variable used by a model. |
| Inference | Running a trained model on new input. |
| API | A defined way for software systems to exchange requests and results. |
| Connector | A packaged integration between Power Platform and another service. |
| Baseline | The simple rule/model or current process used for comparison. |
| Precision / recall | How trustworthy positive alerts are / how many real positives were found. |
| Human in the loop | A person reviews or approves at a defined control point. |

FINAL GATE

# Ten checks before you submit

- 1. The problem, target user, and current baseline are specific.
- 2. The prototype completes one end-to-end job.
- 3. Data source, classification, owner, and license/approval are recorded.
- 4. The method and architecture are understandable to a nontechnical judge.
- 5. Results are compared with a baseline on documented test cases.
- 6. Impact is quantified with visible assumptions.
- 7. AI/compute, human review, licensing, and maintenance costs are included.
- 8. Privacy, legal, compliance, bias, error, and misuse risks have controls.
- 9. Limitations and non-production shortcuts are stated honestly.
- 10. The demo, backup recording, links, and access have been tested.

**The standard**

Build bold - but prove the value, show the controls, and ship something real enough to pilot.

## Source basis

This manual was developed from the NESR AI Prototyping Hackathon 2026 introduction deck, the supplied AI learning-track material, and the Learn AI hackathon guide. Product interfaces and licensing can change; verify availability in the NESR tenant before the event.

OFFICIAL LEARNING LINKS

# Use these when you need the current interface or exact product steps

Microsoft Power Automate documentation
https://learn.microsoft.com/en-us/power-automate/

Power Automate home and getting started
https://learn.microsoft.com/en-us/power-automate/getting-started

Microsoft Power Apps training
https://learn.microsoft.com/en-us/training/powerplatform/power-apps

Sign in to and choose a Power Apps environment
https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/sign-in-to-power-apps

Microsoft Power BI training
https://learn.microsoft.com/en-us/training/powerplatform/power-bi

Google Colab welcome notebook
https://colab.research.google.com/notebooks/intro.ipynb

Kaggle official kagglehub library
https://github.com/Kaggle/kagglehub

pandas getting started
https://pandas.pydata.org/docs/getting_started/index.html

scikit-learn getting started
https://scikit-learn.org/stable/getting_started.html

GitHub documentation
https://docs.github.com/

## Access note

Some services, connectors, AI capabilities, or external sites may be unavailable or restricted by NESR policy, tenant settings, geography, or license. Use approved alternatives and confirm with the relevant administrator. This manual does not replace NESR security, privacy, legal, compliance, records, or AI-use policies.